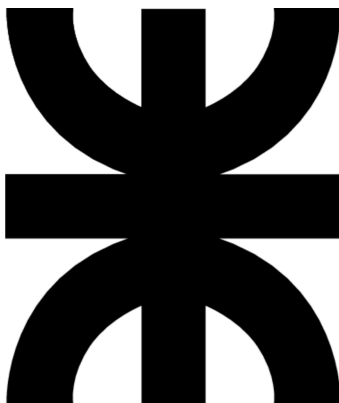


Proyecto Smart Home



Integrantes:

- Fernandez, Orianna Del Rosario
- Aratto, José Alejandro
- Cerqueiro, Carabajal Octavio
- Taus, Fernando Andrés
- Hochmuth, Nehueln Lautaro
- Goy, Bruno Sebastian

Asignatura: Sintaxis y Semántica de los Lenguajes

Carrera: Ing. en Sistemas de Información

Primer cuatrimestre

Curso académico: 2026

Universidad y regional: UTN FRRe

Lugar y fecha: Resistencia, 2026

Índice

1. Introducción.....	3
1.1. Descripción de lo implementado.....	3
1.2. Información y Requerimientos de Software.....	4
2. Conformación de Grupo.....	4
2.1. Integrantes.....	4
2.2. Matriz de Habilidades.....	5
2.3. Identidad.....	5
2.4. Alianza.....	6
3. Gramática.....	6
3.1. Símbolos de la Gramática.....	6
3.1.1. Símbolos Terminales.....	6
3.1.2. Símbolos No Terminales.....	7
3.2. Reglas de Producción.....	8
3.2.1 Regla inicial.....	8
3.2.2 Lista de sentencias.....	8
3.2.3 Tipos de sentencia.....	8
3.2.4 Estructura del bloque WHEN.....	9
3.2.5 Estructura del bloque EVERY.....	9
3.2.6 Estructura del bloque IF.....	9
3.2.7 Reglas de asignación.....	9
3.2.8 Estructura de bloque.....	10
3.2.9 Regla general de condición.....	10
3.2.10 Expresiones lógicas OR.....	11
3.2.11 Expresiones lógicas AND.....	11
3.2.12 Expresiones lógicas NOT.....	11
3.2.13 Condiciones primarias.....	11
3.2.14 Reglas de comparación.....	11
3.2.15 Expresiones temporales.....	13
3.2.16 Operadores.....	13
3.3. Representación Gráfica de Árbol Sintáctico (AST).....	13
4. Análisis Lexico.....	14
4.1 Estructura y Funcionamiento de Lexer.l.....	14
4.1.1. Configuración y Tolerancia (Directivas de Flex).....	15
4.1.2. Definiciones Regulares.....	15
4.1.3. Reconocimiento de Tokens y Lexemas.....	15
4.1.3. Rastreador de Errores (YY_USER_ACTION).....	15
5. Análisis Sintactico.....	15
5.1. Estructura y Funcionamiento del Parser.y.....	16
5.1.1. Declaraciones y Puentes:.....	16
5.1.2. Definición de Tokens.....	16
5.1.3. Definición de Reglas de Producción.....	16
5.1.4. Traducción Dirigida por la Semántica.....	17
5.1.5. Manejo de Errores Sintácticos.....	17

6. Traducción a HTML.....	17
6.1 Implementación.....	17
7. Funciones Auxiliares.....	18
7.1. Módulo Principal main.c.....	18
7.2. Analizador lexico Lexer.l.....	18
7.3. Analizador Sintáctico Parser.y.....	19
8. Modo de Obtención y Estructura de Proyecto.....	19
8.1. Proceso de Construcción.....	19
8.2 Explicación de Archivos fuentes.....	19
9. Modos de Ejecución del Intérprete.....	20
9.1 Modo Interactivo.....	20
9.2 Modo Lectura de Archivo.....	20
10. Conclusiones.....	21
11. Bibliografía.....	21
11.1. Documentación Obligatoria de la Cátedra.....	21
11.2. Software.....	21
11.3. Recursos Audiovisuales (Tutoriales).....	21
11.4. Asistencia por Inteligencia Artificial (IA).....	22
12. Anexos.....	22

1. Introducción

Este trabajo práctico integrador detalla el desarrollo de un compilador/intérprete para un lenguaje de dominio específico orientado a la automatización de hogares inteligentes (Smart Home). El objetivo principal de la herramienta es procesar scripts escritos en este lenguaje personalizado, los cuales los archivos estarán con extensión .smart, y serán traducidos dinámicamente a un panel de control visual en formato HTML.

1.1. Descripción de lo implementado

El trabajo planteado es la implementación de una compilación en dos etapas principales, apoyado por herramientas de generación automática de analizadores y manejo mediante un programa principal en lenguaje C.

- **Analizador Léxico (Flex - Lexer.l)**

Es el encargado de leer letra por letra el archivo que escribimos y agruparlas en "tokens" que el sistema pueda entender. Describimos palabras claves que el analizador deberá reconocer, así como también los nombres de los aparatos como sensores, luces, alarmas, etc. y datos más complejos como horas, fechas o temperaturas. Además, le agregamos un rastreador que anota en qué línea estamos; así, si nos equivocamos al escribir, el programa nos puede decir exactamente en qué renglón está el problema.

- **Analizador Sintáctico y Traductor (Bison - Parser.y)**

Este módulo se encarga de revisar si las palabras que encontró el Lexer forman "oraciones" que tengan sentido. El objetivo es que analice que la construcción de un código siga las reglas establecidas, por ejemplo, que a un WHEN le siga una condición y luego un DO.

En esta parte se hace un trabajo doble: a medida que lee y confirma que una "oración" está bien escrita, automáticamente va traduciendo a un archivo HTML, para tener una visión de lo que se está analizando actuando como un "panel de control".

También incluye un control de errores: si encuentra un error de tipeo o de sintaxis, no deja de analizar instantáneamente, anota el error y sigue revisando el resto del archivo.

- **Controlador Principal (C - main.c):**

Es el programa principal que coordina todo. Primero verifica que el archivo que le pasamos termine en .smart. Si es correcto, crea el archivo web vacío, le pone los estilos visuales básicos y le da la orden al analizador sintáctico de que empiece a traducir. Cuando termina todo el proceso, el analizador nos da un resultado, "0" si el análisis es exitoso y nos genera un HTML para visualizar, pero si encuentra errores en el camino, salta un error de falla de compilación, sin generar el HTML.

1.2. Información y Requerimientos de Software

Para compilar y ejecutar el intérprete de manera correcta, es necesario contar con el siguiente entorno y conjunto de herramientas:

- **Lenguaje Base:** C (Estándar C99 o superior).
- **Generador de Analizador Léxico:** Flex (Fast Lexical Analyzer Generator) - [versión 2.6.4].

- **Generador de Analizador Sintáctico: Bison (GNU Parser Generator)** - [3.8.2].
- **Compilador de C: GCC (GNU Compiler Collection)** - [13.4.0]
- **Entorno de Compilación: Terminal POSIX (Cygwin en Windows / Entorno nativo en Linux)**
- **Editor de Código / IDE: Visual Studio Code.**

Proceso de Compilación

El flujo de compilación se realiza en tres etapas consecutivas:

1. **Análisis Léxico:** Se procesa el análisis léxico con Flex: **flex Lexer.l**
2. **Análisis Sintáctico:** Se genera el analizador sintáctico mediante Bison, utilizando el flag -d para la creación de la cabecera de tokens (Parser.tab.h): **bison -d Parser.y**
3. **Compilación final:** Se compila y vincula el código resultante con main.c utilizando GCC:
 - **En Linux:** `gcc -Wall -o Parser_linux main.c lex.yy.c Parser.tab.c`
 - **En Windows (Cygwin):** `x86_64-w64-mingw32-gcc -Wall -o Parser_win.exe main.c lex.yy.c Parser.tab.c`

Ejecución:

El programa se ejecuta por línea de comandos pasando como único argumento el archivo fuente a analizar (ejemplo estando en la carpeta principal): **./bin/Parser_win.exe ./prueba/ejemplo.smart**

2. Conformación de Grupo

2.1. Integrantes

- Fernandez, Orianna Del Rosario
- Aratto, José Alejandro
- Cerqueiro, Carabajal Octavio
- Taus, Fernando Andrés
- Hochmuth, Nehueln Lautaro
- Goy, Bruno Sebastian

2.2. Matriz de Habilidades

Integrantes	Inglés	Edición de video	Habilidades de Programación	Búsqueda de información	Documentación	Idear soluciones
Aratto José Alejandro	O	★	★	O		★
Taus Fernando Andrés	★	★	★	O	O	★
Hochmuth Nehueln Lautaro	★	O	★	★		O
Goy Bruno Sebastian	★		O	★	O	★
Cerqueiro Carabajal Octavio	★	O	O	★	★	★
Fernandez Orianna Del Rosario	★	★	★		O	O

2.3. Identidad

Nombre: C.mánticos

Logo:



2.4. Alianza

Como grupo, acordamos organizarnos de manera colaborativa y responsable para el desarrollo del trabajo práctico. Nuestro objetivo principal es lograr un proyecto sólido, bien documentado y funcional, distribuyendo las tareas de forma equitativa y aprovechando las fortalezas individuales de cada integrante. Nos proponemos que al menos dos personas del equipo puedan desenvolverse con fluidez en tareas claves como la programación, documentación y planificación del proyecto.

Nos comprometemos a mantener reuniones de trabajo y seguimiento al menos tres veces por semana, priorizando los encuentros nocturnos durante la semana y en la mañana los fines de semana. Estas reuniones se realizan de manera virtual a través de Discord, y utilizaremos GitHub como plataforma principal para el seguimiento del proyecto y mantenerlo siempre actualizado.

Nos comunicaremos de forma constante y fluida por Discord, y en caso de ser necesario, también por otros canales complementarios como WhatsApp.

Todos los integrantes del equipo nos comprometemos a trabajar de forma constante a lo largo del plazo establecido, evitando dejar tareas para último momento, con el objetivo de mantener una buena calidad del trabajo y reducir el estrés asociado a la entrega.

Las metas y valores comunes en nuestro equipo son:

- Responsabilidad individual y grupal.
- Buena comunicación continua.
- Organización y planificación para cumplir en tiempo y forma las entregas.
- Apoyo en dificultades generales.
- Abierto a escuchar opiniones distintas a lo largo del desarrollo y debatir soluciones.

3. Gramática

Para el análisis sintáctico del intérprete Smart-Home, se diseñó una Gramática Libre de Contexto (GLC). Esta gramática define formalmente la estructura y las reglas sintácticas que un script con extensión .smart debe seguir para ser considerado válido.

3.1. Símbolos de la Gramática

3.1.1. Símbolos Terminales

Son los tokens indivisibles identificados por el Analizador Léxico y enviados al Analizador Sintáctico. En nuestra gramática, los representamos convencionalmente en letras mayúsculas. A continuación, se detallan todos los tokens definidos en el sistema agrupados por su funcionalidad:

1. **Palabras reservadas:** Son las sentencias que definen el flujo y comportamiento reactivo del script.
TK_WHEN, TK_IF, TK_THEN, TK_ELSE, TK_DO, TK_END, TK EVERY
2. **Operadores lógicos:** Utilizados para anidar y combinar múltiples condiciones.
TK_AND, TK_OR, TK_NOT

3. Identificadores de Sensores (Entradas):

SENSOR_TEMPERATURA_ID, SENSOR_HUMEDAD_ID, SENSOR_LUZ_ID,
SENSOR_MOVIMIENTO_ID, SENSOR_HUMO_ID, RELOJ_ID

4. Identificadores de Actuadores (Salidas):

FOCO_ID, AIRE_ID, PERSIANA_ID, CERRADURA_ID, ALTAVOZ_ID, ALARMA_ID

5. Atributos de Dispositivos: Definen las propiedades que se pueden leer de los sensores o modificar en los actuadores.

TK_ESTADO, TK_ACTIVADA, TK_BRILLO, TK_COLOR, TK_MODO,
TK_TEMP_OBJ, TK_TEMP_OBJETIVO, TK_TEMP_ACT, TK_POSICION,
TK_VOLUMEN, TK_MUTE, TK_MENSAJE, TK_EMAIL_NOTIF, TK_EMAIL,
TK_HORA, TK_FECHA

6. Operadores y Puntuación: Símbolos matemáticos y estructurales del lenguaje.

- OP_PUNTO (.)
- OP_IGUALDAD (== , !=)
- OP_RELACIONAL (< , > , <= , >=)
- ASIGNACION (=)

7. Literales y Unidades: Valores de datos crudos extraídos por el Lexer y pasados como cadenas (strings) para su uso semántico.

- BOOLEANO (TRUE / FALSE / ON / OFF)
- PORCENTAJE (ej: 80%)
- TEMPERATURA (ej: 24°C)
- ILUMINANCIA (ej: 250lux)
- TIEMPO (ej: 30m)
- VALOR_HORA (ej: 22:00)
- VALOR_FECHA (ej: 21/06/2026)
- EMAIL (ej: usuario@smarthome.com)
- TEXTO (Cadenas entre comillas, ej: "Son las 22hs")
- MODO_AIRE (FRIO/CALOR/VENT)
- VALOR_COLOR (BLANCO / ROJO / AZUL)

3.1.2. Símbolos No Terminales

Son las variables estructurales o abstracciones que se derivan en otras estructuras o tokens. Representan los conceptos del lenguaje y se escriben en minúsculas.

1. Estructura Base:

- programa, lista_sentencias, sentencia, bloque

2. Condicionales y Eventos

- when, every, if_sentencia, if_inicio, then_parte

3. Expresiones Lógicas y condiciones:

- condicion, expresion_or, expresion_and, expresion_not, primaria_condicion

4. Operaciones de Comparación:

- comparacion

- comparacion_temp
- comparacion_humedad
- comparacion_luz
- comparacion_mov
- comparacion_humo
- comparacion_reloj
- comparacion_actuador

5. Operaciones de Asignación:

- asignacion
- asignacion_foco
- asignacion_aire
- asignacion_persiana
- asignacion_cerradura
- asignacion_altavoz
- asignacion_alarma

6. Atributos Adicionales:

- operador_comp

3.2. Reglas de Producción

A continuación se detallan las reglas de derivación de nuestra gramática, utilizando notación. El axioma principal o símbolo inicial de nuestra gramática es programa.

3.2.1 Regla inicial

programa → lista_sentencias

Descripción: Es el punto de entrada principal del compilador. Define que un archivo .smart válido y completo está compuesto fundamentalmente por un conjunto de instrucciones o sentencias.

3.2.2 Lista de sentencias

lista_sentencias → sentencia lista_sentencias

lista_sentencias → sentencia

Descripción: Define la estructura recursiva del código. Permite que el programa procese desde una única instrucción hasta múltiples sentencias concatenadas sin límite de cantidad.

3.2.3 Tipos de sentencia

sentencia → when

sentencia → every

sentencia → if_sentencia

sentencia → asignacion

Descripción: Actúa como un clasificador principal. Establece que cualquier instrucción individual en el script debe pertenecer a una de las cuatro categorías válidas: un evento reactivo (**when**), un ciclo temporal (**every**), un condicional clásico (**if**) o una acción directa de cambio de estado (**asignacion**).

3.2.4 Estructura del bloque WHEN

when → TK_WHEN condicion TK_DO bloque TK_END

Descripción: Define el corazón reactivo del sistema domótico. Espera la palabra reservada **WHEN**, evalúa una condicion lógica y, en caso de cumplirse, ejecuta las acciones contenidas en el bloque, cerrando la estructura con END.

3.2.5 Estructura del bloque EVERY

every → TK EVERY TIEMPO TK_DO bloque TK_END

Descripción: Define una estructura de repetición o cronograma. Permite ejecutar un grupo de acciones (**bloque**) de manera periódica, dictado por el intervalo o momento definido en la variable tiempo.

3.2.6 Estructura del bloque IF

if_sentencia → if_inicio then_parte TK_END

if_sentencia → if_inicio then_parte TK_ELSE bloque TK_END

if_inicio → TK_IF condicion

then_parte → TK_THEN bloque

Descripción: Define las ramificaciones lógicas clásicas. Permite ejecutar un bloque de acciones si la condición es verdadera y provee la capacidad (**opcional**) de ejecutar un bloque alternativo (**ELSE**) en caso de ser falsa.

3.2.7 Reglas de asignación

asignacion → asignacion_foco

asignacion → asignacion_aire

asignacion → asignacion_persiana

asignacion → asignacion_cerradura

asignacion → asignacion_altavoz

asignacion → asignacion_alarma

asignacion_foco → FOCO_ID OP_PUNTO TK_COLOR ASIGNACION VALOR_COLOR

asignacion_foco → FOCO_ID OP_PUNTO TK_ESTADO ASIGNACION BOOLEANO

asignacion_foco → FOCO_ID OP_PUNTO TK_BRILLO ASIGNACION PORCENTAJE

asignacion_aire → AIRE_ID OP_PUNTO TK_TEMP_OBJ ASIGNACION TEMPERATURA

asignacion_aire → AIRE_ID OP_PUNTO TK_TEMP_OBJETIVO ASIGNACION
TEMPERATURA

asignacion_aire → AIRE_ID OP_PUNTO TK_ESTADO ASIGNACION BOOLEANO

asignacion_aire → AIRE_ID OP_PUNTO TK_MODO ASIGNACION MODO_AIRE

asignacion_persiana → PERSIANA_ID OP_PUNTO TK_POSICION ASIGNACION
PORCENTAJE

asignacion_persiana → PERSIANA_ID OP_PUNTO TK_ESTADO ASIGNACION
BOOLEANO

asignacion_cerradura → CERRADURA_ID OP_PUNTO TK_ESTADO ASIGNACION
BOOLEANO

asignacion_altavoz → ALTAVOZ_ID OP_PUNTO TK_VOLUMEN ASIGNACION
PORCENTAJE

asignacion_altavoz → ALTAVOZ_ID OP_PUNTO TK_MUTE ASIGNACION BOOLEANO

asignacion_altavoz → ALTAVOZ_ID OP_PUNTO TK_MENSAJE ASIGNACION TEXTO

asignacion_altavoz → ALTAVOZ_ID OP_PUNTO TK_EMAIL_NOTIF ASIGNACION EMAIL

asignacion_altavoz → ALTAVOZ_ID OP_PUNTO TK_EMAIL ASIGNACION EMAIL

asignacion_alarma → ALARMA_ID OP_PUNTO TK_ESTADO ASIGNACION BOOLAENO

asignacion_alarma → ALARMA_ID OP_PUNTO TK_ACTIVADA ASIGNACION BOOLEANO

Descripción: Determina cómo se modifican los atributos físicos de los actuadores de la casa. Utilizando una sintaxis estructurada con notación de punto, esta regla válida que sólo se asignen valores lógicos a los dispositivos correctos.

3.2.8 Estructura de bloque

bloque → sentencia bloque

bloque → sentencia

Descripción: Similar a lista_sentencias, pero de alcance local. Permite agrupar una o más instrucciones dentro de los "cuerpos" de las estructuras de control (WHEN, EVERY, IF) de manera recursiva.

3.2.9 Regla general de condición

condicion → expresion_or

Descripción: Es el punto de partida para evaluar las reglas lógicas del lenguaje. Delega inmediatamente la evaluación al operador lógico de menor jerarquía (**OR**) para respetar la precedencia correcta.

3.2.10 Expresiones lógicas OR

expresion_or → expresion_and TK_OR expresion_or

expresion_or → expresion_and

Descripción: Maneja la disyunción lógica. Si hay varias condiciones, permite que el bloque se ejecute si al menos una de las ramas es verdadera.

3.2.11 Expresiones lógicas AND

expresion_and → expresion_not TK_AND expresion_and

expresion_and → expresion_not

Descripción: Maneja la conjunción lógica. Tiene mayor prioridad de evaluación matemática que el operador **OR**. Exige que ambas partes de la expresión se cumplan para devolver verdadero.

3.2.12 Expresiones lógicas NOT

expresion_not → TK_NOT expresion_not

expresion_not → primaria_condicion

Descripción: Maneja la negación unitaria. Posee la mayor prioridad entre los operadores lógicos, revirtiendo el valor de verdad de la expresión primaria que le sigue de forma inmediata.

3.2.13 Condiciones primarias

primaria_condicion → comparacion

primaria_condicion → BOOLEANO

Descripción: Representa el nivel más atómico de una expresión lógica. Se resuelve directamente en una comparación física entre dispositivos, o directamente en un literal booleano predefinido (**TRUE** / **FALSE**).

3.2.14 Reglas de comparación

comparacion → comparacion_temp

comparacion → comparacion_humedad

comparacion → comparacion_luz

comparacion → comparacion_mov

comparacion → comparacion_humo

comparacion → comparacion_reloj

comparacion → comparacion_actuador

comparacion_temp → SENSOR_TEMPERATURA_ID operador_comp TEMPERATURA

comparacion_humedad → SENSOR_HUMEDAD_ID operador_comp PORCENTAJE

comparacion_luz → SENSOR_LUZ_ID operador_comp ILUMINANCIA

comparacion_mov → SENSOR_MOVIMIENTO_ID OP_IGUALDAD BOOLEANO

comparacion_humo → SENSOR_HUMO_ID OP_IGUALDAD BOOLEANO

comparacion_reloj → RELOJ_ID OP_PUNTO TK_HORA operador_comp VALOR_HORA

comparacion_reloj → RELOJ_ID OP_PUNTO TK_FECHA operador_comp VALOR_FECHA

comparacion_actuador → FOCO_ID OP_PUNTO TK_ESTADO OP_IGUALDAD
BOOLEANO

comparacion_actuador → FOCO_ID OP_PUNTO TK_BRILLO operador_comp
PORCENTAJE

comparacion_actuador → FOCO_ID OP_PUNTO TK_COLOR OP_IGUALDAD
VALOR_COLOR

comparacion_actuador → AIRE_ID OP_PUNTO TK_ESTADO OP_IGUALDAD BOOLEANO

comparacion_actuador → AIRE_ID OP_PUNTO TK_MODO OP_IGUALDAD MODO_AIRE

comparacion_actuador → AIRE_ID OP_PUNTO TK_TEMP_OBJ operador_comp
TEMPERATURA

comparacion_actuador → AIRE_ID OP_PUNTO TK_TEMP_ACT operador_comp
TEMPERATURA

comparacion_actuador → PERSIANA_ID OP_PUNTO TK_POSICION operador_comp
PORCENTAJE

comparacion_actuador → CERRADURA_ID OP_PUNTO TK_ESTADO OP_IGUALDAD
BOOLEANO

comparacion_actuador → ALTAVOZ_ID OP_PUNTO TK_VOLUMEN operador_comp
PORCENTAJE

comparacion_actuador → ALTAVOZ_ID OP_PUNTO TK_MUTE OP_IGUALDAD
BOOLEANO

comparacion_actuador → ALARMA_ID OP_PUNTO TK_ESTADO OP_IGUALDAD
BOOLEANO

comparacion_actuador → ALARMA_ID OP_PUNTO TK_ACTIVADA OP_IGUALDAD
BOOLEANO

Descripción: Define las lecturas de contexto y estados. Valida sintácticamente las comparaciones entre las métricas ambientales reales captadas por los sensores (temperatura, luz, presencia de humo, hora del reloj) o los estados actuales de los actuadores y los valores definidos en el script, utilizando operadores relacionales (ej. `SENSOR_LUZ_ID < 250lux`).

3.2.15 Operadores auxiliares

operador_comp → OP_IGUALDAD | OP_RELACIONAL

Descripción: Símbolo no terminal auxiliar que unifica los componentes léxicos de equivalencia (**OP_IGUALDAD**) y de magnitud (**OP_RELACIONAL**). Su objetivo es simplificar la sintaxis de la gramática, permitiendo que las reglas de lectura de los sensores y estados de los actuadores acepten cualquier operador lógico-matemático bajo una misma abstracción.

Nota Técnica sobre el alcance Sintáctico: Cabe destacar que la Gramática Libre de Contexto (GLC) aquí definida no incluye reglas de producción para la formación de los símbolos terminales. Esta responsabilidad fue delegada en su totalidad al Analizador Léxico (mediante expresiones regulares), el cual entrega al Analizador Sintáctico los tokens ya conformados como unidades atómicas indivisibles.

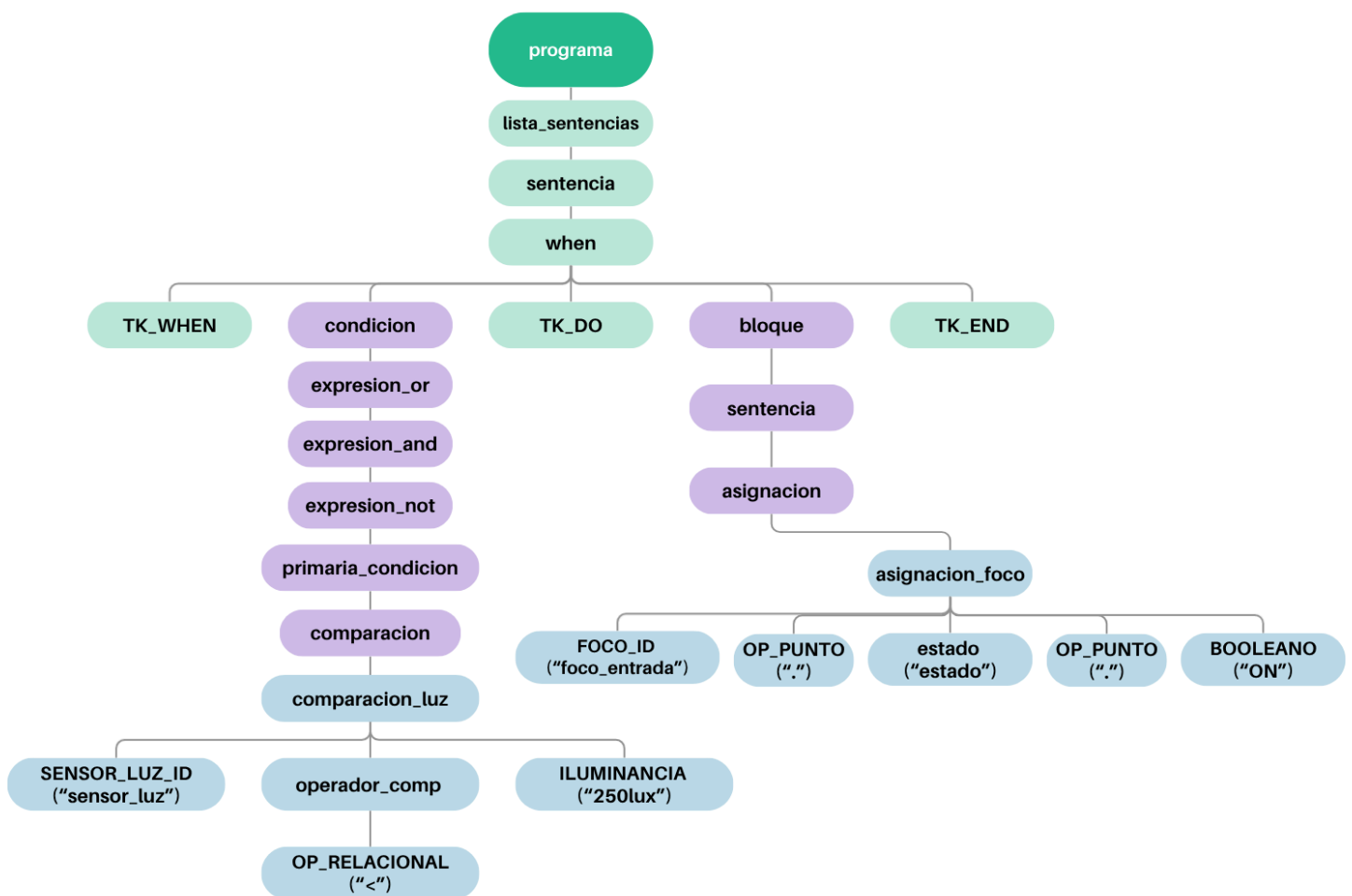
3.3. Representación Gráfica de Árbol Sintáctico (AST)

Un Árbol de Derivación (o Árbol de Análisis Sintáctico) es la representación visual de cómo el Parser agrupa los tokens secuenciales recibidos del Lexer aplicando las reglas de la Gramática Libre de Contexto. En esta estructura, la raíz del árbol es siempre nuestro axioma inicial (programa), las ramas intermedias son los Símbolos No Terminales (las abstracciones lógicas), y las hojas finales son los Símbolos Terminales (los tokens indivisibles).

Para ilustrar el funcionamiento lógico de nuestro intérprete, tomamos como caso de estudio la siguiente sentencia válida del dominio domótico:

WHEN sensor_luz < 250lux DO foco_entrada.estado = ON END

A continuación, se presenta el esquema del árbol sintáctico generado al procesar dicha instrucción, evidenciando la derivación en cascada para la precedencia lógica y la correcta agrupación de los bloques de asignación:



4. Analisis Lexico

El analizador léxico, implementado en el archivo **Lexer.I** utilizando la herramienta Flex, actúa como el "lector" de nuestro intérprete. Su trabajo principal es leer el código fuente escrito por el usuario letra por letra, agrupar esos caracteres en bloques lógicos con significado, llamados tokens, y enviarlos al Analizador Sintáctico.

¿Qué es Flex?

Flex (Fast Lexical Analyzer Generator) es una herramienta de software diseñada para generar analizadores léxicos rápidos y eficientes. Es un programa que escribe otro programa, en nuestro caso código fuente en lenguaje C, capaz de procesar e identificar patrones en un texto.

Flex nos permite trabajar a un nivel más abstracto y declarativo mediante el uso de **Expresiones Regulares**.

4.1 Estructura y Funcionamiento de Lexer.I

Para lograr que nuestro analizador léxico reconozca todos los componentes del lenguaje domótico, estructuramos el archivo en cuatro partes fundamentales:

4.1.1. Configuración y Tolerancia (Directivas de Flex)

Al inicio del archivo, establecimos algunas reglas base para que el intérprete sea amigable con el usuario.

- Utilizamos **%option noyywrap** con esto flex asume que no hay más entrada que procesar al llegar al EOF, termina el programa ahí sin necesidad de definir más funciones.
- Utilizamos **%option case-insensitive** para que el lenguaje no distinga entre mayúsculas y minúsculas (permitiendo que el usuario escriba WHEN, when o wHeN y el sistema lo entienda igual).
- Activamos **%option 8bit** para soportar caracteres especiales, algo indispensable en nuestro dominio para poder leer correctamente el símbolo de grados en las temperaturas.

4.1.2. Definiciones Regulares

Le indicamos al Lexer a reconocer estructuras usando Expresiones Regulares. De esta forma, el sistema atrapa automáticamente formatos complejos como:

- **Magnitudes:** Temperaturas (ej. -5°C), porcentajes (80%) e iluminancia (250lux).
- **Tiempos:** Horas de reloj (22:00), fechas exactas (21/06/2026) y lapsos de tiempo cortos (30m).
- **Datos de usuario:** Direcciones de correo electrónico (usuario@smarthome.com).

4.1.3. Reconocimiento de Tokens y Lexemas

Esta es la parte central del archivo, donde declaramos todo lo que el Analizador Sintáctico está esperando:

- **Palabras reservadas:** Al leer términos de control estructural (WHEN, DO, EVERY), el Lexer simplemente envía una señal de confirmación (devuelve TK_WHEN).
- **Captura de valores (yyival):** Cuando el Lexer encuentra un identificador de un dispositivo (Ej: sensor_luz) o un valor útil (Ej: un color o una temperatura), avisa qué tipo de token encontró y utiliza la función strdup(yytext) para hacer una copia del texto exacto que escribió el usuario. Esta copia es enviada hacia el Parser y es vital para poder generar el diseño HTML final con los datos correctos.

4.1.3. Rastreador de Errores (YY_USER_ACTION)

Cada vez que el Lexer reconoce cualquier texto, sea un token válido o no, guarda ese fragmento en un buffer (linea_actual) que va reconstruyendo el contenido completo de la línea en curso. Si más adelante se detecta un error (léxico o sintáctico), el sistema usa ese buffer para mostrarle al usuario exactamente en qué línea y con qué texto ocurrió el problema.

5. Analisis Sintactico

El analizador sintáctico, desarrollado en el archivo **Parser.y** mediante la herramienta **Bison**, es el "cerebro gramatical" de nuestro proyecto. Su función primordial es recibir el flujo de tokens que le envía Flex y verificar que estén ordenados siguiendo las reglas estrictas de la jerarquía que definimos en nuestra Gramática Libre de Contexto (GLC).

¿Qué es Bison?

Bison es un generador de analizadores sintácticos de propósito general. Mientras que Flex se encarga de agrupar tokens, Bison se encarga de analizar cómo esas palabras se organizan para formar estructuras con sentido.

Bison toma la Gramática Libre de Contexto (GLC) que definimos y genera automáticamente el código en C que implementa el analizador. Este analizador es capaz de verificar si un archivo fuente sigue las reglas gramaticales del lenguaje y permite ejecutar acciones cuando reconoce una estructura específica.

5.1. Estructura y Funcionamiento del Parser.y

Para lograr que nuestro analizador sintáctico reconozca patrones y verificar que estén bien, estructuramos el archivo Parser.y en 4 partes fundamentales:

5.1.1. Declaraciones y Puentes:

En la cabecera del archivo, incluimos las librerías estándares de C necesarias y realizamos el enlace con las variables globales y externas compartidas con el resto del software. Entre ellas, el contador de líneas (**linea**), el acumulador de errores (**cant_errores**), el búfer de texto (**linea_actual**) y el puntero al archivo web de salida (**f_html**). Asimismo, se define la estructura **%union** para establecer que los valores semánticos de los tokens contengan cadenas de caracteres (**char***), permitiendo que los nombres de los sensores y los valores se envíen correctamente.

5.1.2. Definición de Tokens

En esta sección le indicamos a Bison cuáles son las palabras del vocabulario que va a recibir. Declaramos tanto los identificadores de dispositivos (**SENSOR_LUZ_ID**, **FOCO_ID**, **etc.**), como las palabras clave de control estructural (**TK_WHEN**, **TK_DO**, **TK_IF**, **TK EVERY**) y los literales de medición (**TEMPERATURA**, **ILUMINANCIA**, **PORCENTAJE**), asegurando un acoplamiento perfecto con los retornos generados por Flex.

5.1.3. Definición de Reglas de Producción

Es el cuerpo principal del archivo. Aquí se describen las combinaciones válidas del lenguaje. Como el Parser opera mediante un algoritmo de **análisis sintáctico ascendente**, las reglas se van reduciendo secuencialmente desde las estructuras más pequeñas hasta alcanzar el axioma inicial (**programa**).

Un aspecto clave de este diseño fue estructurar las expresiones lógicas de la siguiente forma (**condicion** → **expresion_or** → **expresion_and** → **expresion_not** → **primaria_condicion**). Cada nivel solo combina su propio operador y delega al nivel inferior cualquier sub-expresión aún no resuelta, de modo que el nivel más profundo se evalúa primero. Esto produce automáticamente la precedencia lógica estándar (NOT antes que AND, y AND antes que OR) y garantiza que exista una única forma de derivar cada expresión, eliminando cualquier ambigüedad en el árbol sintáctico.

5.1.4. Traducción Dirigida por la Semántica

El analizador sintáctico además de validar si el código es correcto, funciona también como un traductor activo, esta traducción se denomina **traducción dirigida por la sintaxis**. Para esto, colocamos bloques de código en C nativo encerrados entre llaves "{}" en las reglas de producción, y cada vez que Bison reduce una regla con éxito ejecuta la acción asociada, generando automáticamente bloques de código HTML directamente en el archivo físico de salida (f_html). Si bien en la mayoría de las reglas la acción se ubica al final, en las estructuras de control, Bison permite intercalar acciones intermedias entre los símbolos de la regla, lo que nos permite ir generando las etiquetas HTML de apertura tan pronto como se reconoce cada componente, en lugar de esperar a tener la estructura completa para construir todo el bloque de una sola vez.

Internamente, Bison apila los valores semánticos de cada token (\$1, \$2, \$3...) en una pila paralela a la pila de símbolos, usando el tipo definido en %union. Cuando se dispara una acción, esos \$N quedan disponibles como variables C normales dentro de las llaves, referenciando el valor semántico de cada símbolo de esa regla; en nuestro caso, son char* con los nombres de sensores/actuadores y sus valores asociados, por ejemplo \$1 con el nombre del foco (FOCO_ID) y \$5 con el valor booleano asignado (BOOLEANO), que se insertan directamente en el HTML mediante fprintf.

5.1.5. Manejo de Errores Sintácticos

Para reportar errores de forma clara y específica, configuramos la directiva %define parse.error verbose, que le indica a Bison generar mensajes de error detallados indicando qué token se esperaba en lugar del encontrado. Cuando el analizador detecta un error en el uso de las reglas gramaticales, invoca automáticamente la función yyerror(), donde se incrementa el contador global cant_errores y mostramos al usuario la línea exacta y el

fragmento de código donde ocurrió el problema, reutilizando la información que el Lexer fue acumulando mediante YY_USER_ACTION.

6. Traducción a HTML

Para cumplir con el requerimiento de transformar los archivos.smart a documentos HTML, hemos implementado una estrategia de Traducción Dirigida por la Sintaxis utilizando las acciones semánticas de Bison en el archivo Parser.y.

Esta técnica nos permite integrar bloques de código C directamente en las reglas de producción de nuestra gramática. De esta manera, cada vez que el analizador sintáctico reduce una regla válida, se ejecutan las acciones asociadas que escriben el código HTML correspondiente en un archivo de salida.

6.1 Implementación

La traducción se lleva a cabo mediante 2 módulos importantes:

Gestión de Archivos (main.c): El programa principal es el encargado de inicializar el flujo de escritura del archivo HTML, asignándole el mismo nombre que el archivo fuente (con extensión .html). Además, implementamos una lógica de seguridad que, en caso de detectar errores sintácticos durante el proceso, no se genera el archivo HTML para evitar la entrega de un código incompleto o corrupto.

Acciones Semánticas (Parser.y):

- **Sensores:** Se generan bloques <div> con un borde verde de 1px y padding de 20px. El nombre del sensor y sus valores se inyectan en etiquetas específicas para mantener una estructura semántica.
- **Actuadores:** Se utilizan bloques <div> con borde gris y padding de 20px, estructurando sus atributos internos mediante listas HTML y elementos de lista , garantizando una presentación clara y ordenada.
- **Correos Electrónicos:** En el caso específico del actuador altavoz, los atributos email y email_notif se transforman automáticamente en enlaces hipervinculados utilizando la etiqueta <a> con el protocolo mailto:, permitiendo un contacto directo a través del navegador.

Esta arquitectura es altamente eficiente y escalable. Al inyectar el código HTML durante la reducción sintáctica, minimizamos el uso de memoria adicional, ya que no requerimos mantener un Árbol Sintáctico Abstracto completo en memoria para realizar la traducción. Además, la separación de estilos mediante las configuraciones CSS inyectadas asegura que la salida sea visualmente coherente con los requisitos de la cátedra.

7. Funciones Auxiliares

7.1. Módulo Principal main.c

- **int es_archivo_valido(const char *nombre)**
Valida que el archivo recibido por línea de comandos tenga la extensión .smart. Utiliza **strchr** para ubicar el último punto del nombre y **strcmp** para comparar esa extensión

contra `.smart`, si el archivo no tiene punto o la extensión no coincide, devuelve 0 y el programa rechaza la ejecución antes de intentar abrirlo.

- **int main(int argc, char *argv[])**

Es la parte principal del intérprete. Valida el archivo de entrada, gestiona la apertura del archivo temporal **borrador.tmp** donde Bison escribe la traducción HTML durante el análisis, invoca **yyparse()** para disparar el proceso de análisis léxico-sintáctico, y al finalizar decide si copiar ese contenido al archivo `.html` definitivo, solo si `cant_errores == 0`, pero si tiene errores se descarta el HTML.

También implementa el modo interactivo: cuando se ejecuta sin argumentos, lee sentencias línea por línea desde la consola usando **yy_scan_string()**, las analiza individualmente y va agregando al archivo **Resultado.html** solo las sentencias que resultan sintácticamente válidas.

7.2. Analizador lexico Lexer.l

- **YY_USER_ACTION**

Macro que Flex ejecuta automáticamente antes de cada acción asociada a un patrón reconocido (válido o erróneo). Concatena el lexema actual (**yytext**) al buffer global **linea_actual**, reconstruyendo progresivamente el texto completo de la línea en curso; este registro es el que luego usa **yyerror()** para mostrar el contexto exacto donde ocurrió un error.

- **int yylex(void) (generada automáticamente por Flex a partir de las reglas del archivo)**

Función principal del analizador léxico. Recorre el código fuente carácter por carácter, lo compara contra las expresiones regulares definidas y devuelve al Parser el token correspondiente junto con su valor semántico en **yylval.str** cuando aplica.

7.3. Analizador Sintáctico Parser .y

- **void yyerror(const char *s)**

Función de manejo de errores sintácticos requerida por Bison, invocada automáticamente cuando el flujo de tokens no respeta ninguna regla de la gramática. Incrementa el contador global **cant_errores** e imprime en consola el número de línea y el contenido exacto de esa línea, permitiendo al usuario ubicar y corregir el error con precisión.

- **int yyparse(void) (generada automáticamente por Bison a partir de las reglas del archivo)**

Función principal del analizador sintáctico. Solicita tokens a **yylex()** uno por uno y los va reduciendo según las reglas de producción de la gramática; cada vez que una regla se completa con éxito, ejecuta su acción semántica asociada, que en este proyecto genera el fragmento de HTML correspondiente directamente en **f_html**.

8. Modo de Obtención y Estructura de Proyecto

Para la construcción del intérprete, se ha utilizado una arquitectura basada en herramientas de generación de compiladores (Flex y Bison), lo que permite separar la lógica de análisis léxico de la estructura sintáctica.

8.1. Proceso de Construcción

El intérprete se obtiene mediante un proceso de compilación de tres etapas. Este flujo asegura que, ante cualquier modificación en la gramática o en los patrones léxicos, el sistema pueda regenerarse automáticamente:

1. **Generación del Analizador Léxico:** Se procesa **Lexer.l** con flex para generar el archivo fuente en C **lex.yy.c**.
2. **Generación del Analizador Sintáctico:** Se procesa **Parser.y** con bison (usando el flag -d) para generar **Parser.tab.c** y **Parser.tab.h** (definiciones de tokens compartidas).
3. **Compilación y Enlazado:** Se compilan todos los archivos fuente (**main.c**, **lex.yy.c**, **Parser.tab.c**) utilizando un compilador de C (gcc) para obtener el ejecutable final.

Proceso de Compilación

El flujo de compilación se realiza en tres etapas consecutivas:

1. **Análisis Léxico:** Se procesa el análisis léxico con Flex: **flex Lexer.l**
2. **Análisis Sintáctico:** Se genera el analizador sintáctico mediante Bison, utilizando el flag -d para la creación de la cabecera de tokens (Parser.tab.h): **bison -d Parser.y**
3. **Compilación final:** Se compila y vincula el código resultante con main.c utilizando GCC:
 - En Linux: **gcc -Wall -o Parser_linux main.c lex.yy.c Parser.tab.c**
 - En Windows (Cygwin): **x86_64-w64-mingw32-gcc -Wall -o Parser_win.exe main.c lex.yy.c Parser.tab.c**

8.2 Explicación de Archivos fuentes

El proyecto está compuesto por los siguientes archivos fuentes:

Archivo	Rol	Descripción
main.c	Driver	Es el punto de entrada del programa. Gestiona la validación del archivo de entrada (.smart), inicializa el puntero global f_html y orquesta la ejecución del yyparse().
Lexer_parser.l	Especificación Léxica	Contiene las reglas para identificar cada token del lenguaje (palabras reservadas, sensores, actuadores, unidades). Define la estructura del autómata léxico.

lex.yy.c	Fuente del Lexer	Archivo generado automáticamente. Código fuente C generado por Flex a partir de Lexer_parser.l, que implementa la función yylex().
Parser.y	Definición Gramatical	Contiene la Gramática Libre de Contexto (GLC). Incluye las reglas de producción y las acciones semánticas en C que ejecutan la traducción a HTML.
Parser.tab.h	Cabecera de Tokens	Archivo generado automáticamente por Bison. Contiene las definiciones numéricas de los tokens, permitiendo que tanto el Lexer como el Parser compartan un lenguaje común.
Parser.tab.c	Fuente del parser	Archivo generado automáticamente. Código fuente C generado por Bison a partir de Parser.y, que implementa la función yyparse() y contiene las acciones semánticas ya traducidas a C nativo.

9. Modos de Ejecución del Intérprete

9.1 Modo Interactivo

Este modo es obligatorio y fundamental para la etapa del Analizador Léxico. Permite procesar la entrada de forma directa desde la consola.

Rol en el Lexer: Es la herramienta de validación técnica por excelencia. Permite ingresar lexemas individuales o sentencias cortas para verificar que el autómata manual los clasifique correctamente en tiempo real. Es indispensable durante el desarrollo para asegurar que no existan errores en las transiciones de estado.

Rol en el Parser: Se mantiene disponible como funcionalidad de soporte. Es ideal para probar rápidamente reglas gramaticales específicas o fragmentos sintácticos aislados sin la necesidad de escribir y gestionar archivos físicos.

Ejemplos de ejecución: (estando en la carpeta principal)

Comando de ejecución en Windows:

- .\bin\lexer_win.exe
- .\bin\Parser_win.exe

Comando de ejecución en Linux:

- ./bin/lexer_linux
- ./bin/Parser_linux

9.2 Modo Lectura de Archivo

Este modo es el estándar operativo y es el más importante para la etapa del Analizador Sintáctico y la Traducción a HTML.

Rol en el Parser/Traductor: Es el modo de producción. Dado que el objetivo final es la generación de un archivo .html a partir de un script .smart completo, este modo permite procesar grandes volúmenes de reglas lógicas. Aquí es donde la gramática se despliega en su totalidad y el traductor genera la estructura HTML completa.

Rol en el Lexer: Está presente para realizar pruebas de carga y verificar el comportamiento del lexer frente a archivos de prueba largos, asegurando que el reporte de errores y el conteo de líneas sean consistentes en archivos reales.

Ejemplos de ejecución: (estando en la carpeta principal)

Comando de ejecución en Windows (CMD / PowerShell):

- `.\bin\Lexer_win.exe .\pruebalejemplo.smart`
- `.\bin\Parser_win.exe .\pruebalejemplo.smart`

Comando de ejecución en Linux:

- `./bin/Lexer_linux ./prueba/ejemplo.smart`
- `./bin/Parser_linux ./prueba/ejemplo.smart`

10. Conclusiones

Luego de un gran camino de investigación, desarrollo, prueba y error que tuvimos que atravesar para llevar a cabo este proyecto llegamos a un punto final.

Este trabajo nos enseñó muchas herramientas que se nos eran desconocidas como flex o bison, investigamos e implementamos como definir expresiones regulares, las palabras reservadas y demás definiciones que necesitábamos para nuestra gramática. También como definir las reglas de la gramática y el método de traducción directamente desde el parser.

Aprendimos el manejo de terminales en distintos entornos ya que como creamos ejecutables para linux y windows debimos investigar cómo tener una máquina virtual de Ubuntu en windows usando WSL.

Más allá de todas las dificultades se logró desarrollar un intérprete funcional con todas las indicaciones propuestas por la cátedra.

11. Bibliografía

Para la realización de este Trabajo Práctico Integrador, se han consultado diversas fuentes documentales, recursos audiovisuales y herramientas de asistencia técnica que permitieron la comprensión de los conceptos teóricos y la implementación del intérprete.

11.1. Documentación Obligatoria de la Cátedra

Documentación técnica provista por el equipo docente para el desarrollo del proyecto:

- UTN FRRE. (2026). SSL2026-TPI_InterpreteSmartHome.pdf: *Especificaciones técnicas y requerimientos del TPI*. Cátedra de Sintaxis y Semántica de los Lenguajes.

11.2. Software

Recursos utilizados para la descarga de herramientas:

- Cygwin (2026). Entorno de desarrollo y herramientas GCC para Windows. Disponible en: <https://www.cygwin.com/install.html>

11.3. Recursos Audiovisuales (Tutoriales)

Videos consultados para la instalación, configuración y resolución de errores comunes:

- [Fantasma del teclado](#) (2024). Cómo instalar Flex y Bison en Windows || How to install Flex and Bison. <https://www.youtube.com/watch?v=nO4Sla3pe0I>
- Dami AI Studio (2021). FLEX Analizador Lexico - Explicacion, Compilacion y Pruebas. <https://www.youtube.com/watch?v=AyB7gVNor9U&t=283s>
- Benjamin Escobar (2025). Analizador Léxico con Flex. <https://www.youtube.com/watch?v=114Pt0mG2nc&t=467s>
- [Universitat Politècnica de València - UPV](#) (2011). Bison. Desarrollo de un analizador sintáctico usando Bison | 5/13 | UPV. <https://www.youtube.com/watch?v=vfDLKxbTGQo>
- [Platzi](#) (2024). Linux en Windows y Windows en Linux (Tutorial WSL). <https://www.youtube.com/watch?v=Qy44XLpiChc&t=236s>

11.4. Asistencia por Inteligencia Artificial (IA)

Se declara el uso de herramientas de IA como colaboradores en el proceso de desarrollo:

- **Google Gemini (2026)**. IA colaboradora. Utilizada como tutor técnico y asistente en la revisión de código, optimización de la gramática (Parser.y), depuración de errores lógicos en el autómata (Lexer.l) y asistencia en el proceso de investigación para la traducción a HTML.
- **Google NotebookLM (2026)**. Herramienta de gestión de contexto. Utilizada para organizar, sintetizar y realizar consultas sobre la documentación oficial de la cátedra, facilitando la extracción de requerimientos específicos del TPI.

12. Anexos